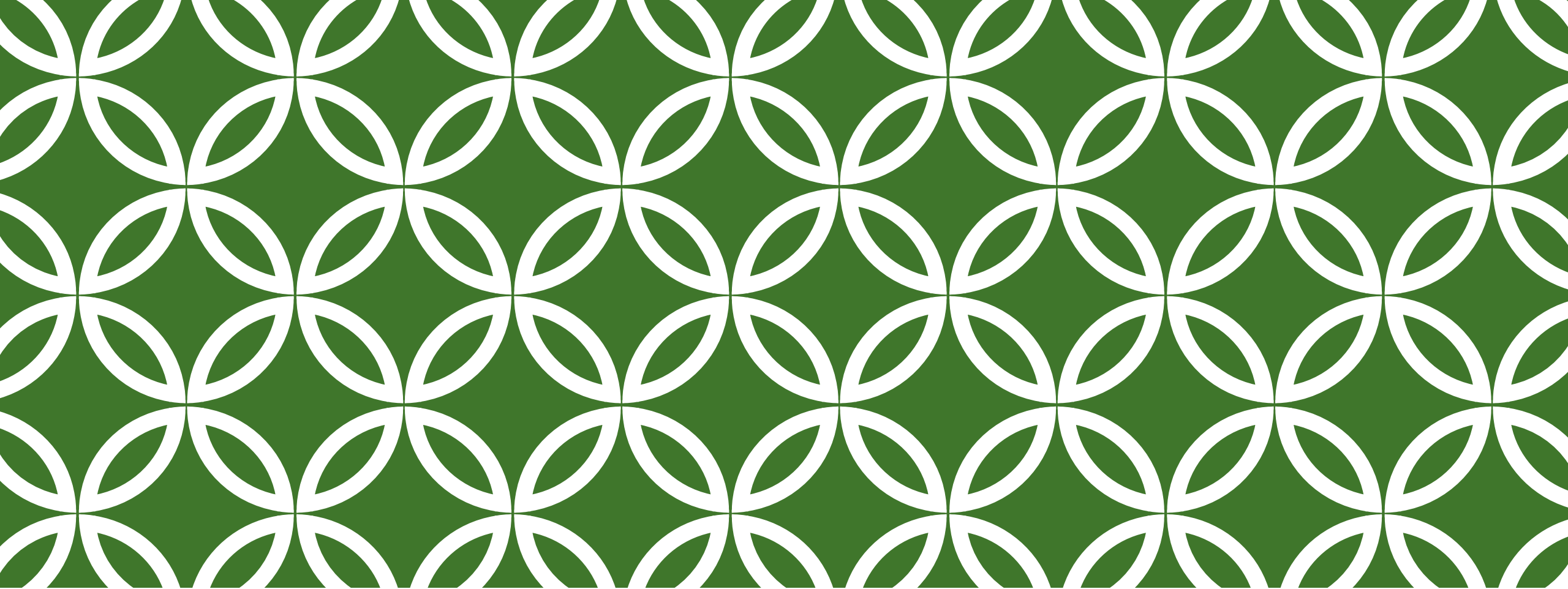




ФУНКЦИОНАЛЬНАЯ ПАРАДИГМА ЯЗЫКОВ ПРОГРАММИРОВАНИЯ

Дисциплина Языки и системы
программирования
(Парадигмы языков
программирования)

Литовка Наталья Васильевна

ЧАСТИЧНОЕ ПРИМЕНЕНИЕ ФУНКЦИЙ

Частичное применение — возможность в ряде языков программирования зафиксировать часть аргументов многоместной функции и создать другую функцию, меньшей аргументности. Например, имея трёхместную функцию $f: (X \times Y \times Z) \rightarrow N$, можно зафиксировать («привязать») первый аргумент $x_0 \in X$, и построить бинарную функцию $f_{x_0}: (Y \times Z) \rightarrow N$ такую, что $f_{x_0}(y, z) = f(x_0, y, z)$ для любых $y \in Y, z \in Z$.

В практике программирования частичное применение используется в связи с тем, что функции, полученные путём фиксации некоторого количества аргументов, зачастую оказываются полезными. К примеру, многие языки программирования имеют функцию или оператор инкремента (являющимся фиксацией единицы как одного из аргументов двухместного сложения) или функцию обращения значения. Частичное применение возможно реализовать практически во всех языках программирования, обладающих возможностью определения многоместных функций. В ряде языков есть специализированные конструкции для частичного применения, например, в стандартной библиотеке C++ имеется функция *bind*, которая на входе принимает имя функции и аргументы, и возвращает функциональный объект — результат частичного применения переданного набора аргументов к функции.

В ряде функциональных языков программирования, таких как ML и Haskell, функции объявляются в каррированной форме по умолчанию, то есть, понятие о многоместности функций реализуется средствами одноместных функций высших порядков; в этих условиях частичным применением (называемом в этих языках сечением операторов) оказывается всякая передача неполного числа аргументов.

В языках программирования с функциями первого класса можно определить *curry*, *uncurry* и *rapply*, чтобы осуществить каррирование и частичное применение напрямую (при этом связь между ними следующая: *curry rapply = curry*). Но такой подход может привести к большему времени выполнения программы в связи с созданием дополнительных замыканий, в то время как Haskell позволяет использовать более эффективные приёмы.

КОМПОЗИЦИЯ ФУНКЦИЙ

Композиция функций (или суперпозиция функций) — это применение одной функции к результату другой.

Композиция функций G и F обычно обозначается $G \circ F$, что обозначает применение функции G к результату функции F , то есть $(G \circ F)(x) = G(F(x))$.

Определение

Пусть $F: X \rightarrow Y$ и $G: F(X) \rightarrow Z$ — две функции ($F(X) \subset Y$). Тогда их композицией называется функция $G \circ F: X \rightarrow Z$, определённая равенством:

$$(G \circ F)(x) = G(F(x)), x \in X$$

Связанные определения

Термин «сложная функция» может быть применен к композиции двух функций, тем не менее он чаще употребляется в ситуации, когда на вход функции нескольких переменных подаётся сразу несколько функций от одной или нескольких исходных переменных. Например, сложной можно назвать функцию G вида

$$G(x, y) = F(u(x, y), v(x, y)),$$

потому что она представляет собой функцию F , которой на вход подаются результаты функций u и v .

Свойства композиции

Композиция ассоциативна:

$$(H \circ G) \circ F = H \circ (G \circ F).$$

Если $F = id_X$ — тождественное отображение на X , то есть

$$F(x) = id_X(x) = x, \forall x \in X,$$

то

$$G \circ id_X = G$$

Если $G = id_Y$ — тождественное отображение на Y , то есть

$$G(y) = id_Y(y) = y, \forall y \in Y,$$

то $id_Y \circ F = F$.

Рассмотрим пространство всех биекций множества X на себя и обозначим его $\mathcal{F}_X$. То есть, если $F \in \mathcal{F}_X$, то $F: X \rightarrow X$ — биекция. Тогда композиция функций в $\mathcal{F}_X$ является бинарной операцией, а $(\mathcal{F}_X, \circ)$ — группой. id_X является нейтральным элементом этой группы. Обратным к элементу $F \in \mathcal{F}_X$ является $F^{-1} \in \mathcal{F}_X$ — обратная функция.

Группа $(\mathcal{F}_X, \circ)$, вообще говоря, не коммутативна, то есть $F_1 \circ F_2 \neq F_2 \circ F_1$.

КАРРИРОВАНИЕ ФУНКЦИЙ

Каррирование — преобразование функции от многих аргументов в набор функций, каждая из которых является функцией от одного аргумента. Возможность такого преобразования впервые отмечена в трудах Готтлоба Фреге, систематически изучена Моисеем Шейнфинкелем в 1920-е годы, а наименование получило по имени Хаскелла Карри — разработчика комбинаторной логики, в которой сведение к функциям одного аргумента носит основополагающий характер.

Определение:

Для функции двух аргументов $h: (A \times B) \rightarrow C$ оператор каррирования Λ выполняет преобразование $\Lambda(h): A \rightarrow (B \rightarrow C)$ — берёт аргумент типа A и возвращает функцию типа $(B \rightarrow C)$. С интуитивной точки зрения, каррирование функции позволяет фиксировать её некоторый аргумент, возвращая функцию от остальных аргументов. Таким образом, Λ — функция $(A \times B \rightarrow C) \rightarrow (A \rightarrow (B \rightarrow C))$.

Каррирование широко используется в языках программирования, прежде всего, поддерживающих парадигму функционального программирования. В некоторых языках функции каррированы по умолчанию, то есть, многоместные функции реализуются как одноместные функции высших порядков, а применение аргументов к ним — как последовательность частичных применений.

В языках программирования с функциями первого класса обычно определены операции *curry* (переводящая функцию сигнатуры вида $A, B \rightarrow C$ в функцию сигнатуры $A \rightarrow B \rightarrow C$) и *uncurry* (осуществляющее обратное преобразование — ставящая в соответствие функции сигнатуры вида $A \rightarrow B \rightarrow C$ двуместную функцию вида $A, B \rightarrow C$). В этих случаях прозрачна связь с операцией частичного применения *apply*: *curry apply = curry*.

ОПЕРАЦИИ ИЗМЕНЕНИЯ ПОРЯДКА ВЫЧИСЛЕНИЙ

При выполнении сразу нескольких арифметических операций следует учитывать порядок их выполнения. Приоритет операций от наивысшего к низшему:

Инкремент, декремент

Умножение, деление, получение остатка

Сложение, вычитание

Для изменения порядка следования операций применяются **скобки**.

Ветвление...

Цикл...

АНОНИМНЫЕ ФУНКЦИИ

Анонимная функция в программировании — особый вид функций, которые объявляются в месте использования и не получают уникального идентификатора для доступа к ним. Поддерживаются во многих языках программирования.

Обычно при создании анонимные функции либо вызываются напрямую, либо ссылка на функцию присваивается переменной, с помощью которой затем можно косвенно вызывать данную функцию. Но в последнем случае анонимная функция получает имя и уже перестаёт быть анонимной. Если анонимная функция ссылается на переменные, не содержащиеся в её теле (захват), то такая функция называется замыканием. Лямбда-выражение — типичная для многих языков синтаксическая конструкция для определения анонимной функции.

Синтаксис записи анонимных функций для различных языков программирования в большинстве случаев сильно различается.

СПИСКИ

В информатике, **список** (*list*) — это абстрактный тип данных, представляющий собой упорядоченный набор значений, в котором некоторое значение может встречаться более одного раза.

Списки в функциональных языках являются фундаментальной структурой. Большинство функциональных языков имеет встроенные средства для работы со списками вроде получения длины списка, головы (первый элемент списка), хвоста (часть списка, идущая за первым элементом), применения функции к каждому элементу списка (Map), свертки списка и пр.

АНАМОРФИЗМ

Анаморфизм — понятие из теории категорий, имеющее непосредственное применение в функциональном программировании. Является одним из базовых примитивов для описания рекурсивных функций. Совместно с сопутствующими понятиями катаморфизма, хиломорфизма и параморфизма может использоваться для представления произвольных рекурсивных функций. Однако, принимая во внимание высокую степень абстракции теории категорий, понятие катаморфизма можно применять в тех областях научного знания, где имеется необходимость в применении примитивов для рекурсии. В том же функциональном программировании данное понятие можно использовать не только для функций, но и в механизме вывода типов (например, в рамках модели статической типизации Хиндли — Милнера).

В функциональном программировании анаморфизм является обобщением разворачивания (для списков) на произвольные алгебраические типы данных, что может быть описано при помощи терминальных коалгебр.

Одна из первых публикаций на эту тему в контексте программирования — статья «Functional Programming with Bananas, Lenses, Envelopes and Barbed Wire» («Функциональное программирование при помощи бананов, линз, конвертов и колючей проволоки»), которая описывала анаморфизм на языке программирования Squiggol.

КАТАМОРФИЗМ

Катаморфизм — понятие из теории категорий, имеющее непосредственное применение в функциональном программировании. Является одним из базовых примитивов для описания рекурсивных функций (рекурсивных процессов). Совместно с сопутствующими понятиями анаморфизма, хиломорфизма и параморфизма может использоваться для представления произвольных рекурсивных функций. Однако, принимая во внимание высокую степень абстракции теории категорий, понятие катаморфизма можно применять в тех областях научного знания, где имеется необходимость в применении примитивов для рекурсии. В том же функциональном программировании данное понятие можно использовать не только для функций, но и в механизме вывода типов (например, в рамках модели статической типизации Хиндли — Милнера), равно как и при кодогенерации и вычислении результатов в процессе интерпретации программ.

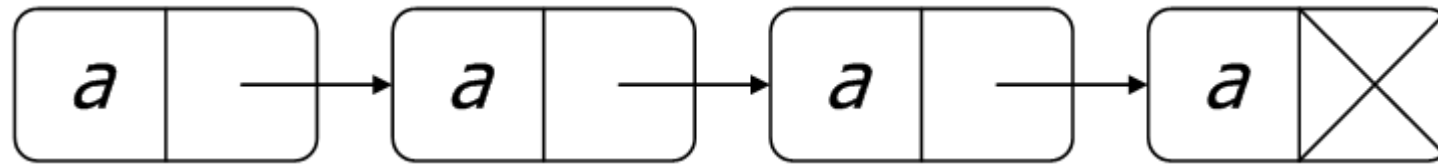
В функциональном программировании катаморфизм является обобщением свёртки (для списков) на произвольные алгебраические типы данных, что может быть описано при помощи начальных алгебр (понятие из теории категорий).

Одна из первых публикаций на эту тему в контексте программирования — статья «Functional Programming with Bananas, Lenses, Envelopes and Barbed Wire» («Функциональное программирование при помощи бананов, линз, конвертов и колючей проволоки»), которая описывала катаморфизмы в стиле Squiggol.

Пример

Наиболее просто катаморфизм можно понять при помощи примеров. Наиболее простым является катаморфизм на обыкновенных линейных списках (т. н. префиксированных списках). От простого к сложному: далее катаморфизм объясняется при помощи линейного списка и бинарных деревьев. Далее приводятся поясняющие примеры, написанные при помощи функционального языка программирования Haskell.

Список



Пусть имеются следующие определения специального синонима для представления алгебр над списками (`ListAlgebra`), функции для организации свёртки списка (`foldList`) и нескольких примеров утилитарных функций, основанных на функции свёртки.


```
type ListAlgebra a r = (r, a -> r -> r)

foldList :: ListAlgebra a r -> [a] -> r
foldList (f, g) []      = f
foldList (f, g) (x:xs) = g x (foldList (f, g) xs)

listLength :: ListAlgebra a Integer
listLength = (0, \x xs -> 1 + xs)

listSum :: Num a => ListAlgebra a a
listSum = (0, \x xs -> x + xs)
```

В приведённом примере функция `foldList (f, g)` является катаморфизмом для типа `[a]`, а функции `listLength` и `listSum` называются алгебрами.

Более подробные и многочисленные примеры функций, основанных на использовании катаморфизма для списка, находятся здесь: [примеры для списка](#).

ССЫЛКИ

[Функциональное программирование](#)

[Функциональное программирование с точки зрения EcmaScript.](#)

[Каррирование](#)

[Частичное применение](#)

[Композиция функций](#)

[Анонимная функция](#)

[Анаморфизм](#)

[Катаморфизм](#)